DOMAIN DRIVEN DESIGN - JAVA - GLOBAL SOLUTION 2026



NexRain

Monitoramento inteligente de enchentes e alagamentos urbanos

Documentação do Projeto

Equipe NexRain

Mateus Saavedra De Mendonça Cruz - RM563266

Davi Melo - RM561818

Danilo Fernandes - RM561657

FIAP - Engenharia de Software - Global Solution 2026 (2º semestre) - CitizenTech

Sumário

1. Descritivo da Solução 3

2. Justificativa 3

3. Objetivos 4

4. Principais Funcionalidades e Arquitetura 4

5. Tabela de Endpoints 6

    5.1 Exemplos de requisição 8

6. Protótipos de Tela 9

7. Diagrama Entidade-Relacionamento (DER) 13

8. Diagrama de Classes 14

    8.1 Camada de domínio (entities e api) 14

    8.2 DTOs e exceções 15

    8.3 Camadas Controller, Service e DAO 16

    8.4 Serviços de integração, IA e infraestrutura 17

9. Como Executar o Projeto 18

1. Descritivo da Solução

O **NexRain** é uma plataforma de monitoramento inteligente voltada à identificação e à prevenção dos riscos causados por enchentes e alagamentos urbanos. A solução reúne em um só lugar três fontes de informação que hoje ficam espalhadas: as **imagens das câmeras da cidade**, a **previsão de chuva em tempo real** e os **relatos dos próprios cidadãos**.

A parte desenvolvida na disciplina de Domain Driven Design é a **API RESTful em Java com Spring Boot**, que concentra as regras de negócio da solução e guarda as informações no banco de dados Oracle da FIAP. Ela é consumida por um site em Next.js (publicado na Vercel), enquanto a própria API roda no Render.

O funcionamento acompanha os quatro pilares definidos pela equipe:

- **Monitoramento por IA:** a imagem de uma câmera urbana (ou a foto enviada por um cidadão) é analisada por um modelo de visão computacional (OpenAI GPT-4o-mini). O modelo informa se há alagamento, estima a altura da água e se a via ainda é transitável. Sem a chave da OpenAI, a API usa um motor local próprio que analisa cor e textura da imagem.
- **Análise e registro:** cada alagamento detectado vira uma *ocorrência* com localização, horário, altura da água, nível de risco e confiança. A ocorrência fica *pendente* até um operador da Defesa Civil confirmar, descartar (falso positivo) ou marcar como resolvida.
- **Alertas aos cidadãos:** ao confirmar uma ocorrência de risco alto ou crítico a API emite um alerta automaticamente. A previsão de chuva da API Open-Meteo também é cruzada com o histórico de cada bairro para gerar alertas preventivos, antes da água subir.
- **Mobilidade inteligente:** um algoritmo de roteamento consulta o OSRM (OpenStreetMap), verifica quais caminhos passam pelo raio de algum alagamento ativo e, quando necessário, cria pontos de desvio dos dois lados da rua alagada até encontrar uma rota segura.

Por ser um protótipo da Global Solution, as imagens das câmeras vêm de bases públicas (Wikimedia Commons, licenças Creative Commons) e existe um modo de **temporal simulado** que soma uma chuva forte à previsão real, para demonstrar o comportamento do sistema mesmo em dias secos. Todas as demais informações (previsão, radar, mapas e rotas) vêm de APIs públicas reais.

2. Justificativa

Os alagamentos são um dos problemas urbanos mais recorrentes das grandes cidades brasileiras. Em São Paulo, chuvas de verão de poucos minutos são suficientes para interditar avenidas como a 23 de Maio, a Marginal Tietê ou a região da Praça da Bandeira, causando congestionamentos, perda de veículos e, nos casos mais graves, risco de vida para quem tenta atravessar uma rua tomada pela água.

Hoje a informação chega ao cidadão de forma tardia e fragmentada: a previsão do tempo fala da cidade inteira, os alertas oficiais não dizem qual rua evitar e os aplicativos de trânsito só percebem o alagamento depois que os carros já pararam. Ao mesmo tempo, a cidade já possui milhares de câmeras instaladas que poderiam ser usadas para identificar o problema automaticamente.

> O NexRain se justifica por transformar dados que já existem (câmeras, previsão do tempo e mapas abertos) em uma resposta prática para a pergunta que o cidadão realmente faz: **“por onde eu passo agora sem cair num alagamento?”**.

3. Objetivos

Objetivo geral

Desenvolver uma API RESTful em Java com Spring Boot e banco Oracle que represente o domínio de monitoramento de alagamentos urbanos e sirva de base para identificar, registrar, validar e comunicar riscos de enchente, além de sugerir rotas que evitem as vias comprometidas.

Objetivos específicos

- Analisar imagens de câmeras urbanas com visão computacional e estimar a altura da água e o nível de risco.
- Registrar as ocorrências no Oracle com localização, horário, risco estimado e o fluxo de validação pela Defesa Civil.
- Calcular um índice de risco (0 a 100) por região cruzando a previsão real de chuva, o limite de chuva que o bairro suporta, o histórico de alagamentos e as ocorrências ativas.
- Emitir alertas automáticos por ocorrência confirmada e por previsão de chuva forte, sem duplicar alertas.
- Calcular rotas de carro que desviam dos pontos alagados, comparando tempo e distância com a rota mais rápida.
- Oferecer CRUD completo (GET, POST, PUT e DELETE) de regiões, câmeras, ocorrências, alertas e rotas, com validações e tratamento de erros padronizado.
- Publicar a API no Render e o site na Vercel, ambos funcionando com dados reais.

4. Principais Funcionalidades e Arquitetura

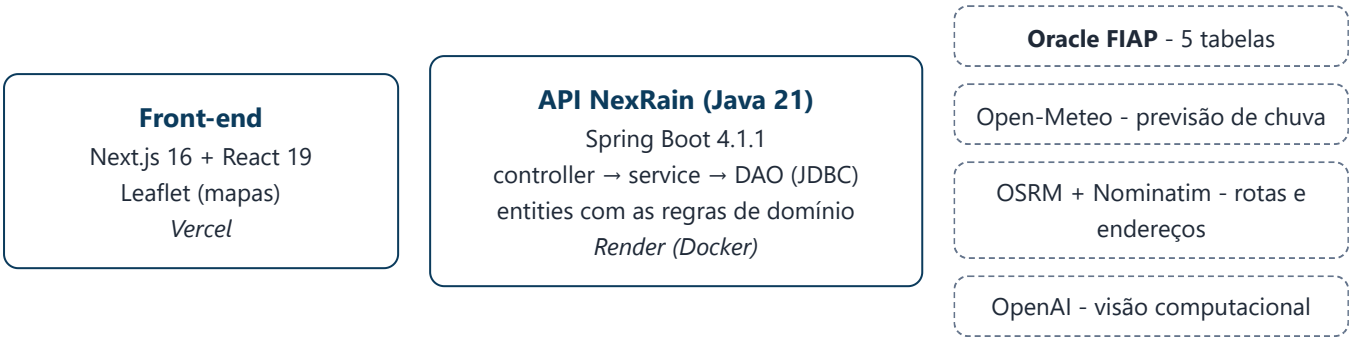
Visão computacional
Análise da câmera ou da foto do cidadão com GPT-4o-mini Vision ou com o motor local (cor e textura). Uma mesma câmera não gera ocorrências duplicadas: a ocorrência ativa é atualizada.

Índice de risco por região
Regra de negócio da classe `MonitorRisco`: intensidade da chuva x limite do bairro, chuva acumulada, probabilidade, histórico e ocorrências ativas.

Ciclo de vida da ocorrência
Pendente → confirmada / descartada → resolvida. As regras ficam na própria entidade `Ocorrencia` (não é possível confirmar algo já resolvido, por exemplo).

Rota segura
Algoritmo da classe `PlanejadorRota`: custo = tempo de viagem + penalidade por alagamento no caminho; gera pontos de desvio perpendiculares à rua e descarta desvios com "ida e volta".

Arquitetura e integrações



O radar meteorológico em tempo real (RainViewer) e o mapa base (Esri / OpenStreetMap) são carregados direto pelo navegador.

Organização do código (Domain Driven Design)

O projeto segue a mesma organização em camadas dos projetos anteriores da disciplina, com as regras de negócio dentro das classes de domínio e não espalhadas pelos controllers:

Pacote	Responsabilidade
`entities`	Domínio: `Regiao`, `Camera`, `Ocorrencia` (ciclo de vida da validação), `Alerta`, `Rota`, o objeto de valor `Coordenada` (Haversine, distância até trecho de rua) e as regras `MonitorRisco` (índice de risco) e `PlanejadorRota` (algoritmo de desvio).
`services`	Casos de uso e validações (CRUD de cada recurso, monitoramento por IA, risco, rotas) e as integrações com Open-Meteo, OSRM/Nominatim e OpenAI.
`dao`	Acesso ao Oracle com JDBC puro (`PreparedStatement`), um DAO por tabela.
`controller`	Endpoints REST, configuração de CORS e o `TratadorExcecoes` (respostas de erro padronizadas).
`api` / `dto`	Objetos que vêm das APIs externas (previsão, trajetos, análise da IA) e objetos de entrada/saída da API.
`conexoes` / `excecoes`	`ConexaoFactory`, `Configuracao` (lê variáveis de ambiente ou `local.properties`), `ClienteHttp` e as exceções de negócio.
`main`	Classes de teste pelo console (`TesteConexao`, `TesteSelecionarRegiao`, `TesteMonitorRisco`).

Regra do índice de risco (MonitorRisco)

Fator	Pontos (máx.)
Chuva máxima prevista em 6h dividida pelo limite de chuva do bairro	até 50
Chuva acumulada nas próximas 3h	até 15
Probabilidade de chuva	até 10
Suscetibilidade do bairro (ALTA 20, MEDIA 10, BAIXA 0; metade se não houver chuva)	até 20
Ocorrências ativas na região (crítica 30, alta 20, média 10, baixa 4)	até 35

Índice 0-24 = BAIXO · 25-49 = MÉDIO · 50-74 = ALTO · 75-100 = CRÍTICO. Para a altura da água: abaixo de 10 cm BAIXO, 10-29 cm MÉDIO, 30-49 cm ALTO (carro de passeio não passa) e a partir de 50 cm CRÍTICO.

5. Tabela de Endpoints

Base local: `http://localhost:8080` · Base publicada: `https://nexrain-springboot.onrender.com`. Todas as requisições e respostas usam JSON.

Verbo	URI	Descrição	Status
Monitoramento, previsão e mapas			
POST	`/api/monitoramento/analises`	Analisa a imagem de uma câmera ou foto (visão computacional) e registra a ocorrência	200, 201, 400, 404, 502, 500
GET	`/api/riscos?simulacao=false`	Risco de alagamento de cada região (previsão real ou temporal simulado)	200, 502, 500
GET	`/api/clima?latitude=&longitude=`	Clima atual e chuva prevista hora a hora de um ponto	200, 400, 502
GET	`/api/geocodificacao?endereco=`	Busca coordenadas de um endereço (OpenStreetMap)	200, 400, 502
GET	`/api/painel`	Números do painel (regiões, câmeras, ocorrências, alertas, rotas)	200, 500
GET	`/api/health`	Verifica se a API está no ar e qual motor de IA está ativo	200
Regiões			
GET	`/api/regioes`	Lista as regiões monitoradas	200, 500
GET	`/api/regioes/{id}`	Busca uma região pelo id	200, 400, 404, 500
GET	`/api/regioes/{id}/cameras`	Câmeras da região	200, 400, 404, 500
GET	`/api/regioes/{id}/ocorrencias`	Histórico de ocorrências da região	200, 400, 404, 500
POST	`/api/regioes`	Cadastra uma região	201, 400, 500
PUT	`/api/regioes/{id}`	Atualiza uma região	200, 400, 404, 500
DELETE	`/api/regioes/{id}`	Exclui uma região (e suas câmeras, ocorrências e alertas)	204, 400, 404, 500
Câmeras			
GET	`/api/cameras`	Lista as câmeras	200, 500
GET	`/api/cameras/{id}`	Busca uma câmera pelo id	200, 400, 404, 500
POST	`/api/cameras`	Cadastra uma câmera	201, 400, 404, 500
PUT	`/api/cameras/{id}`	Atualiza uma câmera	200, 400, 404, 500
DELETE	`/api/cameras/{id}`	Exclui uma câmera	204, 400, 404, 500
Ocorrências			
GET	`/api/ocorrencias?status=&ativas=`	Lista as ocorrências (filtros opcionais por status ou só ativas)	200, 400, 500
GET	`/api/ocorrencias/{id}`	Busca uma ocorrência pelo id	200, 400, 404, 500

Verbo	URI	Descrição	Status
POST	`/api/ocorrencias`	Registra um alagamento (relato do cidadão ou operador)	201, 400, 404, 500
PUT	`/api/ocorrencias/{id}`	Atualiza uma ocorrência	200, 400, 404, 500
PUT	`/api/ocorrencias/{id}/validacao`	Confirma, descarta ou resolve a ocorrência (gera/encerra alerta)	200, 400, 404, 500
DELETE	`/api/ocorrencias/{id}`	Exclui uma ocorrência	204, 400, 404, 500
Alertas			
GET	`/api/alertas?vigentes=`	Lista os alertas (todos ou só os em vigor)	200, 500
GET	`/api/alertas/{id}`	Busca um alerta pelo id	200, 400, 404, 500
POST	`/api/alertas`	Emite um alerta manual	201, 400, 404, 500
POST	`/api/alertas/previsao?simulacao=`	Gera alertas para as regiões com risco alto ou crítico	200, 201, 502, 500
PUT	`/api/alertas/{id}`	Atualiza (ou encerra) um alerta	200, 400, 404, 500
DELETE	`/api/alertas/{id}`	Exclui um alerta	204, 400, 404, 500
Rotas seguras			
GET	`/api/rotas`	Histórico de rotas calculadas	200, 500
GET	`/api/rotas/{id}`	Busca uma rota com o trajeto completo	200, 400, 404, 500
POST	`/api/rotas`	Calcula a rota que desvia dos alagamentos e salva	201, 400, 502, 500
PUT	`/api/rotas/{id}`	Recalcula uma rota salva com os alagamentos atuais	200, 400, 404, 502, 500
DELETE	`/api/rotas/{id}`	Exclui uma rota	204, 400, 404, 500

200 sucesso · 201 criado · 204 excluído · 400 dados inválidos ou regra de negócio violada · 404 não encontrado · 502 API externa (clima, mapas, imagem) indisponível · 500 erro no banco. Os erros sempre voltam no formato `{"status": 400, "erro": "mensagem"}`.

5.1 Exemplos de requisição

POST /api/rotas - calcular rota segura

```
// requisição (origem e destino podem ser endereço em texto ou coordenadas)
{ "origem": "Praça da Sé", "origemLat": -23.5503, "origemLon": -46.6339,
  "destino": "Shopping Ibirapuera", "destinoLat": -23.61, "destinoLon": -46.666 }

// resposta 201 (trechos)
{ "rota": { "id": 8, "situacao": "SEGURA", "distanciaKm": 9.2, "duracaoMin": 17.9,
            "distanciaDiretaKm": 9.4, "duracaoDiretaMin": 13.9, "minutosExtras": 4.0,
            "alagamentosEvitados": 1, "alagamentosNoTrajeto": 0,
            "trajeto": [ { "latitude": -23.55031, "longitude": -46.63386 }, ... ] },
  "alagamentosNoCaminhoDireto": [ { "id": 3, "endereco": "Av. 23 de Maio - altura do Viaduto Tutoia",
"nivelRisco": "ALTO", ... } ],
  "alternativasAvaliadas": 4,
  "mensagem": "Rota segura encontrada: desvia de 1 alagamento(s) com 4.0 min a mais de viagem." }
```

POST /api/monitoramento/analises - analisar câmera

```
{ "idCamera": 1, "registrarOcorrencia": true }
// também aceita uma foto: { "imagemBase64": "...", "latitude": -23.56,
"longitude": -46.65, "endereco": "..." }

// resposta 201
{ "analise": { "alagamentoDetectado": true, "nivelAguaEstimadoCm": 35, "nivelRisco": "ALTO", "confianca":
69.0,
               "viaTransitavel": false, "descricao": "...", "origem": "motor-local" },
  "camera": { "id": 1, "nome": "CAM-01 Praça da Bandeira", ... },
  "ocorrencia": { "id": 1, "status": "PENDENTE", "origem": "IA_CAMERA", ... },
  "mensagem": "Alagamento detectado e registrado como ocorrência #1, aguardando validação." }
```

PUT /api/ocorrencias/{id}/validacao - validar ocorrência

```
{ "acao": "CONFIRMAR", "responsavel": "Defesa Civil - Operador 07" }   // ou DESCARTAR / RESOLVER

// resposta 200: a ocorrência com status CONFIRMADA. Se o risco for ALTO ou CRITICO um alerta é criado.
// resposta 400 ao confirmar de novo:
{ "status": 400, "erro": "Só é possível confirmar uma ocorrência pendente (status atual: CONFIRMADA)" }
```

POST /api/ocorrencias - relato do cidadão

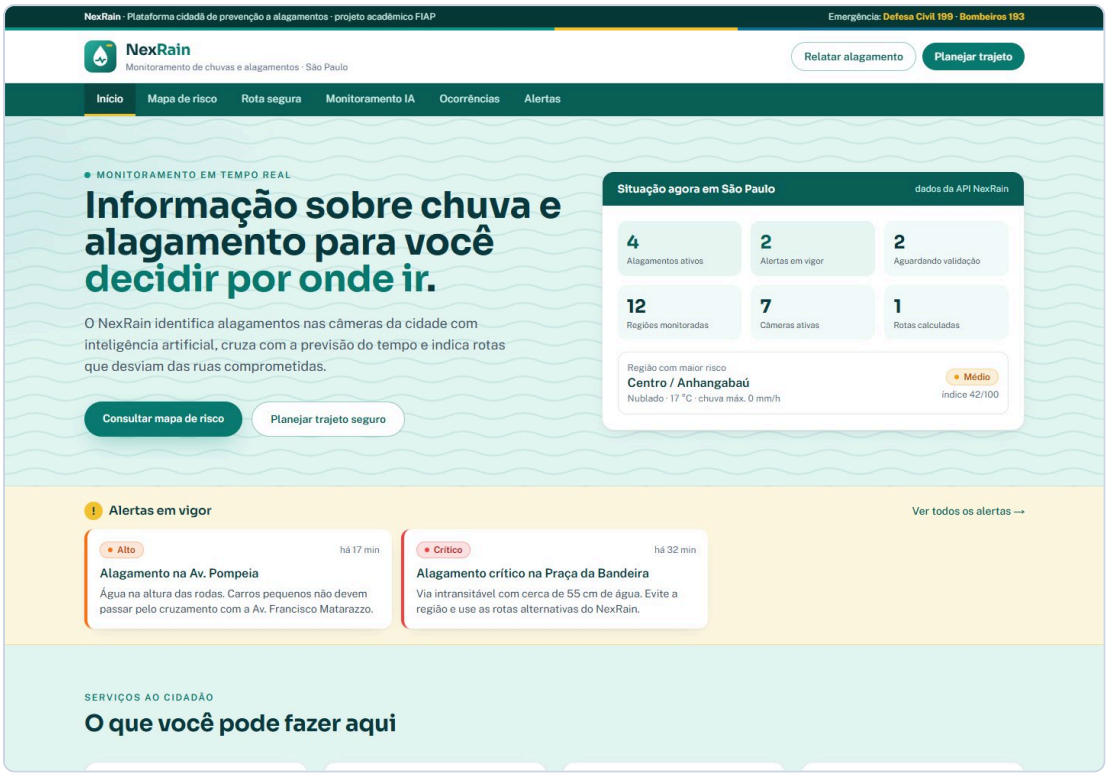
```
{ "endereco": "Av. Prestes Maia", "latitude": -23.5418, "longitude": -46.6355, "nivelAguaCm": 60,
  "descricao": "Água acima da roda dos carros" }
// sem idRegiao a API vincula a região mais próxima; sem nivelRisco ele é calculado pela altura da água
(60 cm = CRITICO)
```

GET /api/riscos?simulacao=false - risco por região (trecho)

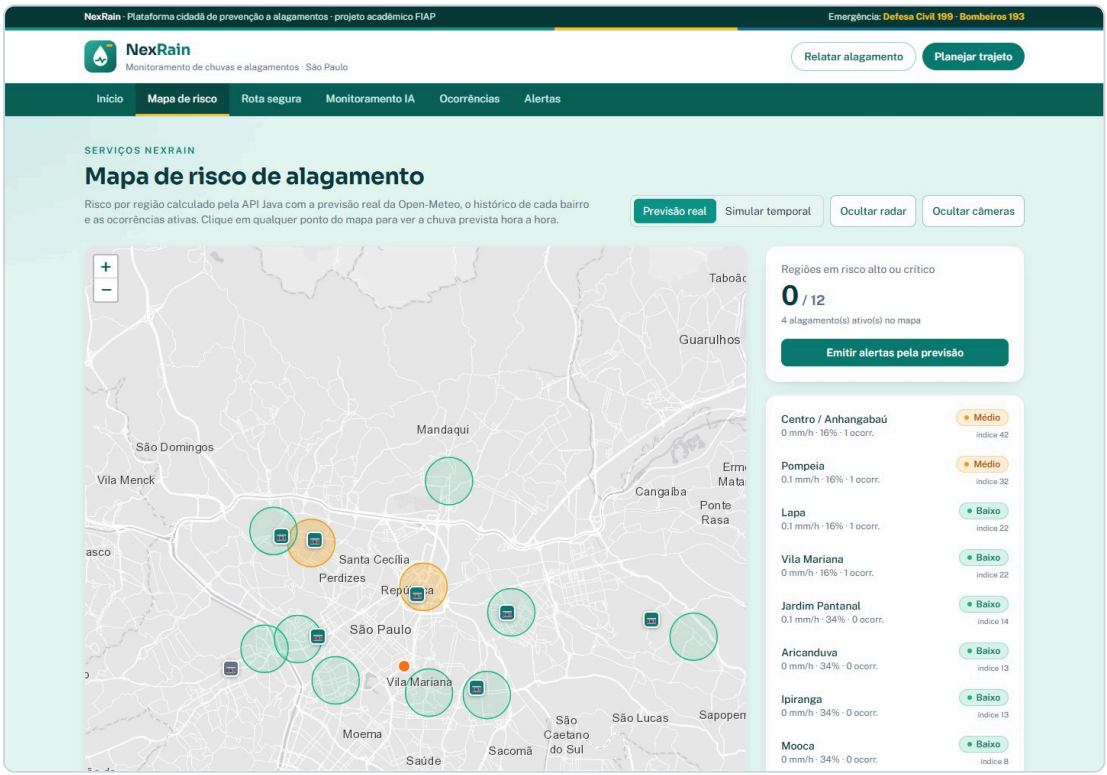
```
[ { "regiao": { "id": 1, "nome": "Centro / Anhangabaú", "suscetibilidade": "ALTA", "limiteChuvaMm": 20.0,
... },
    "indice": 42, "nivel": "MEDIO", "chuvaMaxima6h": 0.0, "probabilidadeChuva": 16, "descricaoTempo":
"Nublado",
    "fatores": [ "Sem chuva significativa prevista nas próximas 6h", "Bairro com histórico frequente de
alagamentos",
                 "2 ocorrência(s) de alagamento ativa(s) na região" ],
    "recomendacao": "Atenção a pontos de alagamento. ...", "simulado": false }, ... ]
```

6. Protótipos de Tela

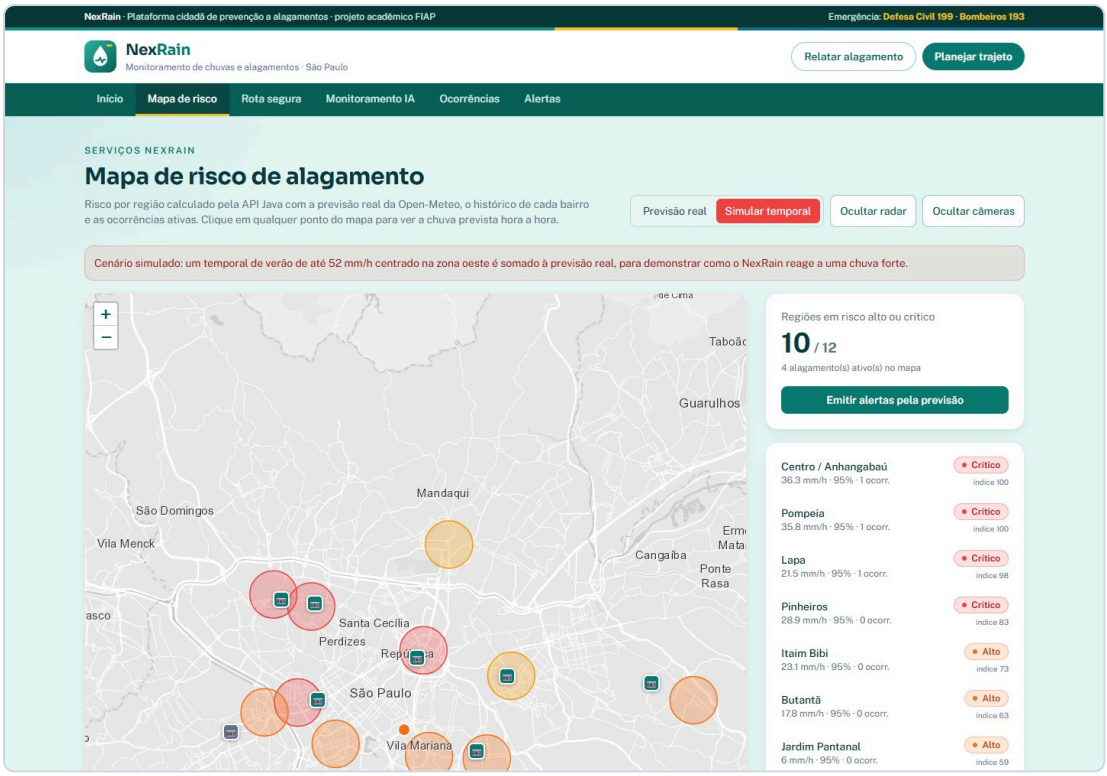
Telas do front-end em Next.js, já integradas à API (capturas do sistema rodando com dados reais de previsão e mapas).



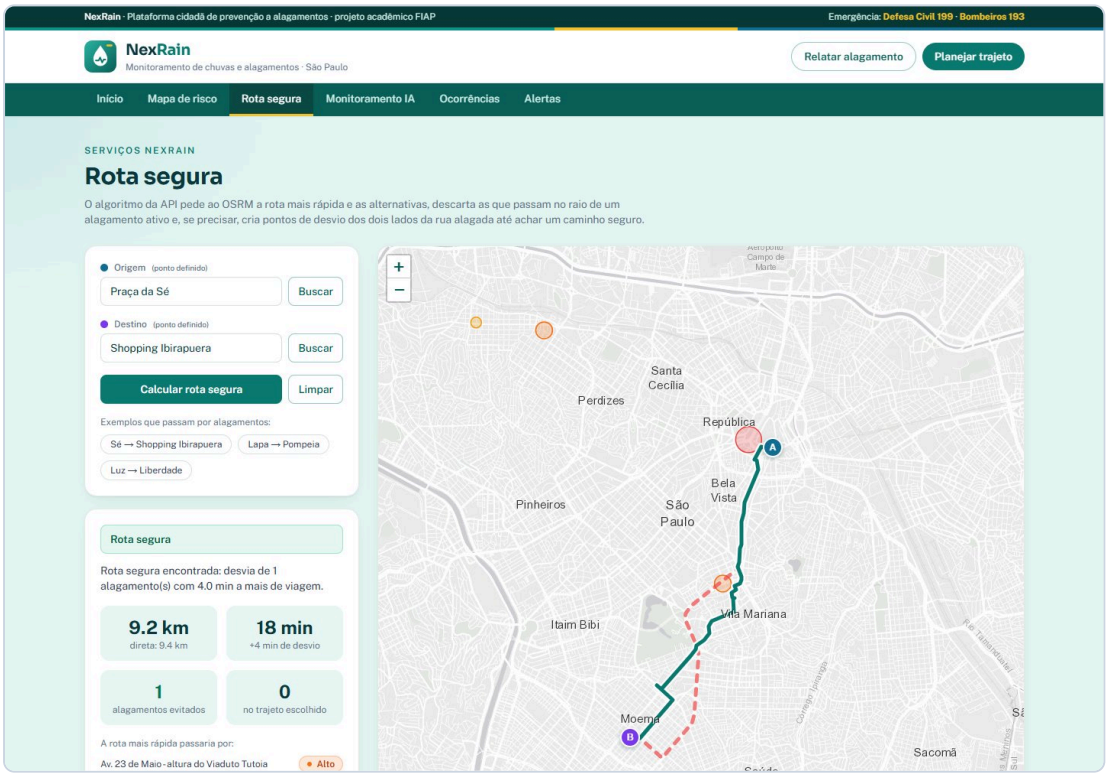
Início - painel com os números da API (`GET /api/painel`), alertas em vigor (`GET /api/alertas?vigentes=true`) e os quatro pilares da solução.



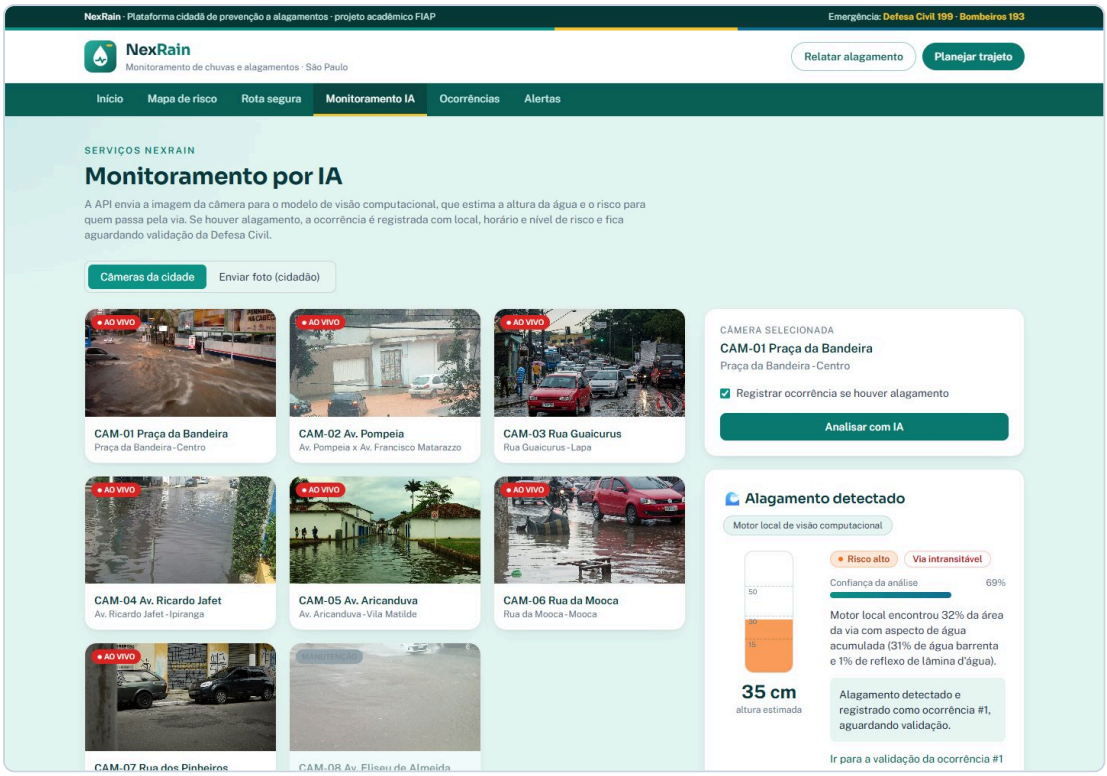
Mapa de risco (previsão real) - regiões pintadas pelo índice de risco (`GET /api/riscos`), câmeras, ocorrências ativas e radar de chuva. Clicar no mapa mostra a chuva prevista hora a hora (`GET /api/clima`).



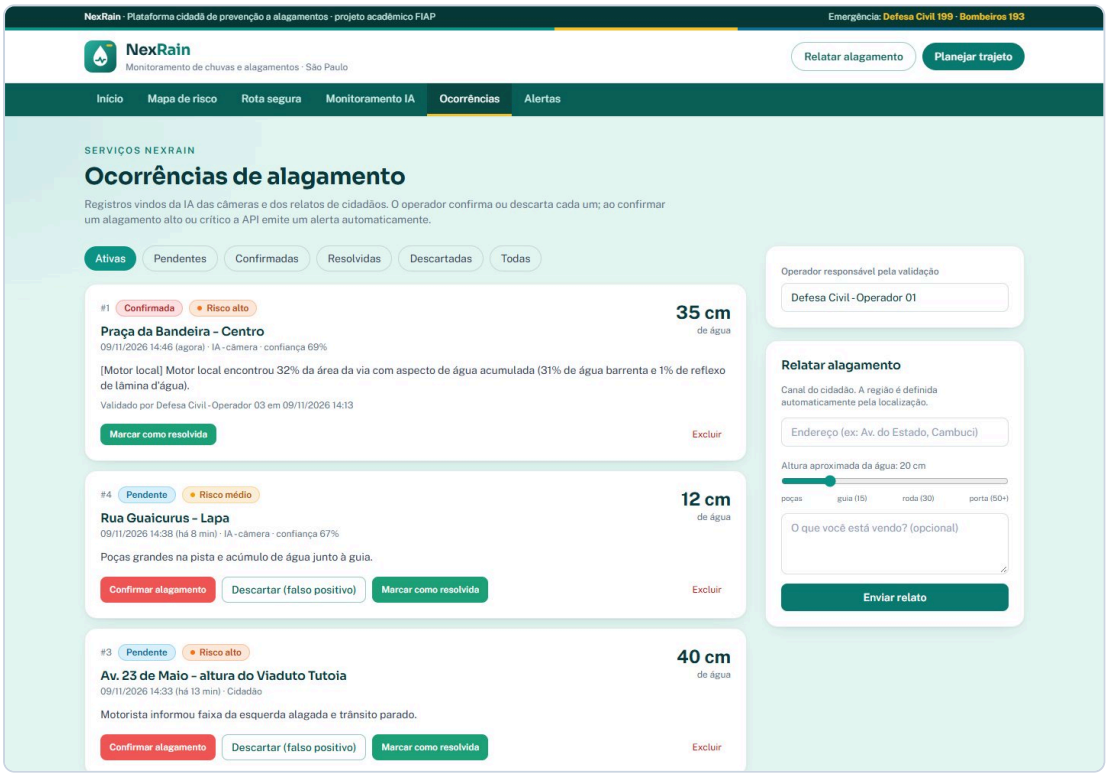
Mapa de risco (temporal simulado) - mesmo cálculo com uma chuva forte somada à previsão (`GET /api/riscos?simulacao=true`). O botão "Emitir alertas pela previsão" chama `POST /api/alertas/previsao`.



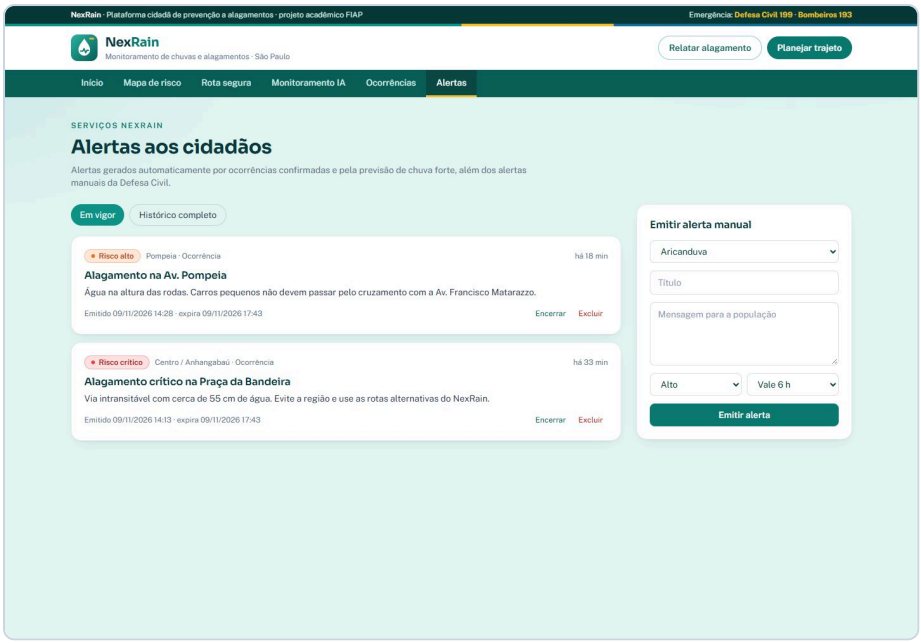
Rota segura - em vermelho tracejado a rota mais rápida, que passaria pelo alagamento da Av. 23 de Maio; em verde a rota calculada pelo NexRain (`POST /api/rotas`). Abaixo, o histórico com as ações de ver, recalcular (`PUT`) e excluir (`DELETE`).



Monitoramento por IA - câmeras cadastradas e o resultado da análise da CAM-01 (`POST /api/monitoramento/analises`): alagamento de 35 cm, risco alto, via intransitável e ocorrência registrada para validação.



Ocorrências - fila de validação da Defesa Civil (`PUT /api/ocorrencias/{id}/validacao`) e formulário de relato do cidadão (`POST /api/ocorrencias`).



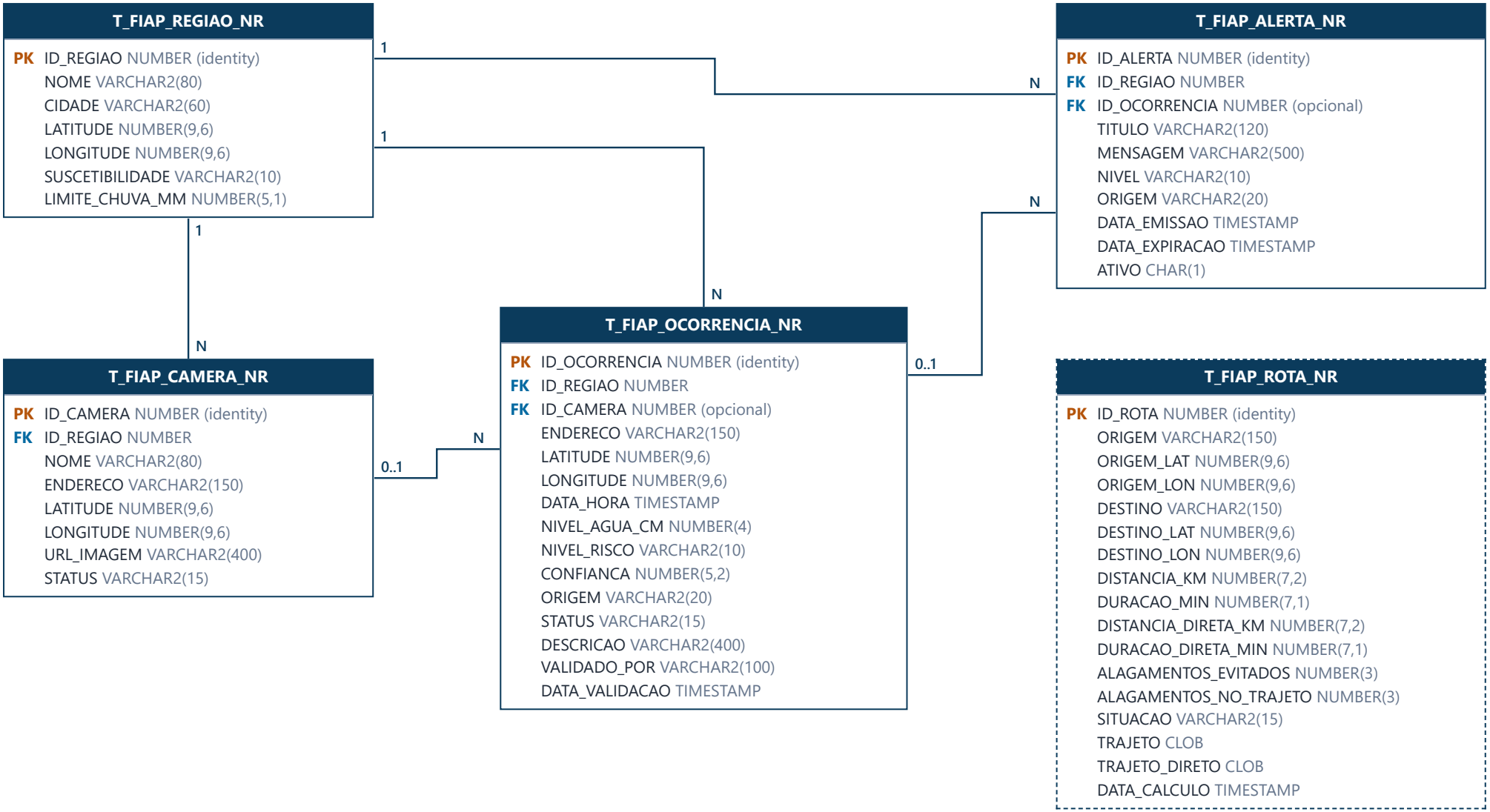
Alertas - alertas em vigor e histórico, com encerrar/reativar (`PUT`), excluir (`DELETE`) e emissão manual (`POST /api/alertas`).



Versão celular - o site é responsivo; aqui a rota segura no celular.

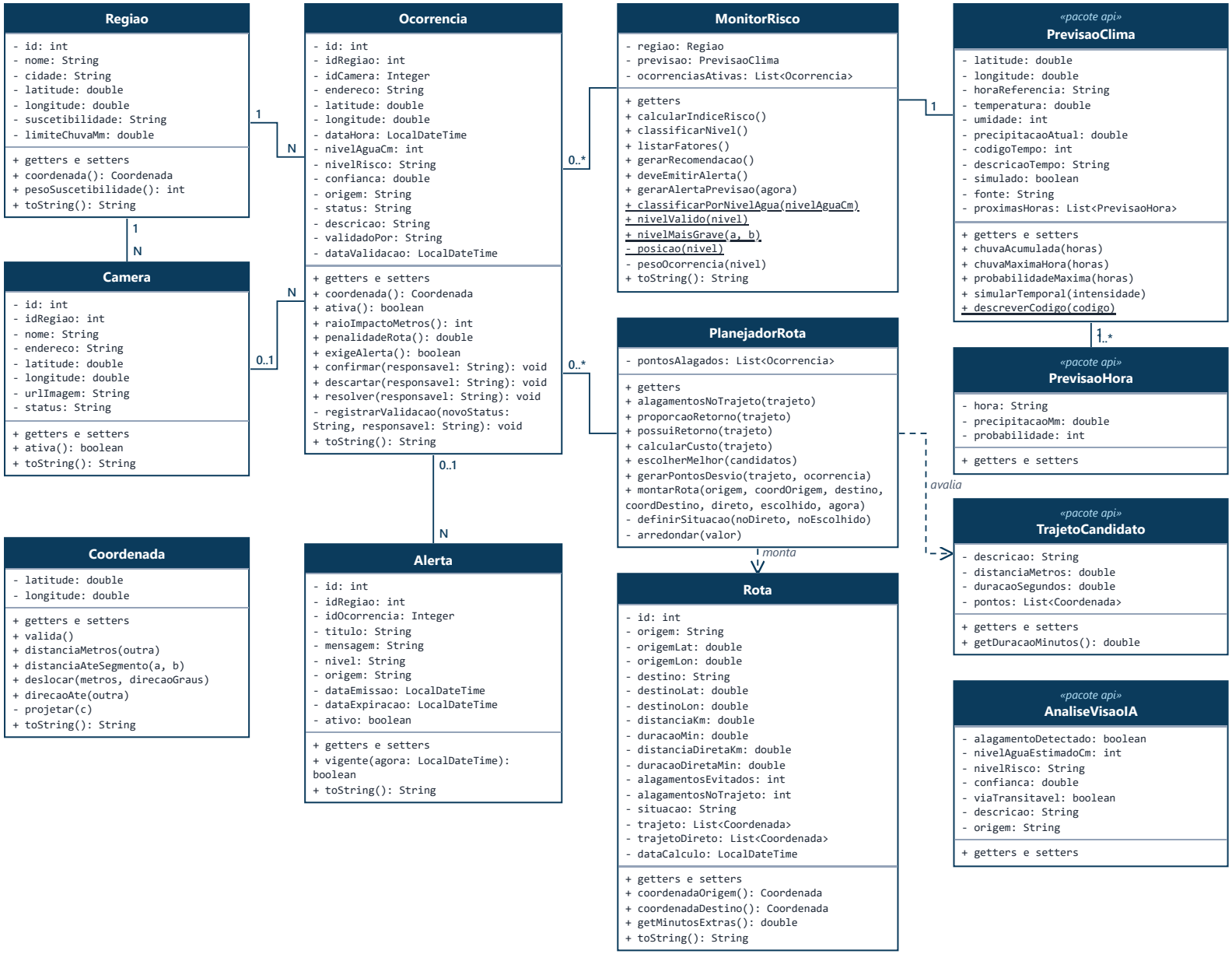
7. Diagrama Entidade-Relacionamento (DER)

Cinco tabelas no Oracle. Uma região tem várias câmeras, ocorrências e alertas; uma câmera pode gerar várias ocorrências; uma ocorrência pode originar alertas. A tabela de rotas guarda o histórico do algoritmo e não tem chave estrangeira (os alagamentos evitados são calculados no momento do cálculo).



8. Diagrama de Classes

8.1 Camada de domínio (pacotes `entities` e `api`). Atributos e métodos extraídos do código-fonte. `MonitorRisco` e `PlanejadorRota` concentram as regras de negócio; `Coordenada` é um objeto de valor usado por `Rota.trajeto`, `TrajetoCandidato.pontos` e pelos métodos `coordenada()`. `MonitorRisco.gerarAlertaPrevisao()` cria um `Alerta`.



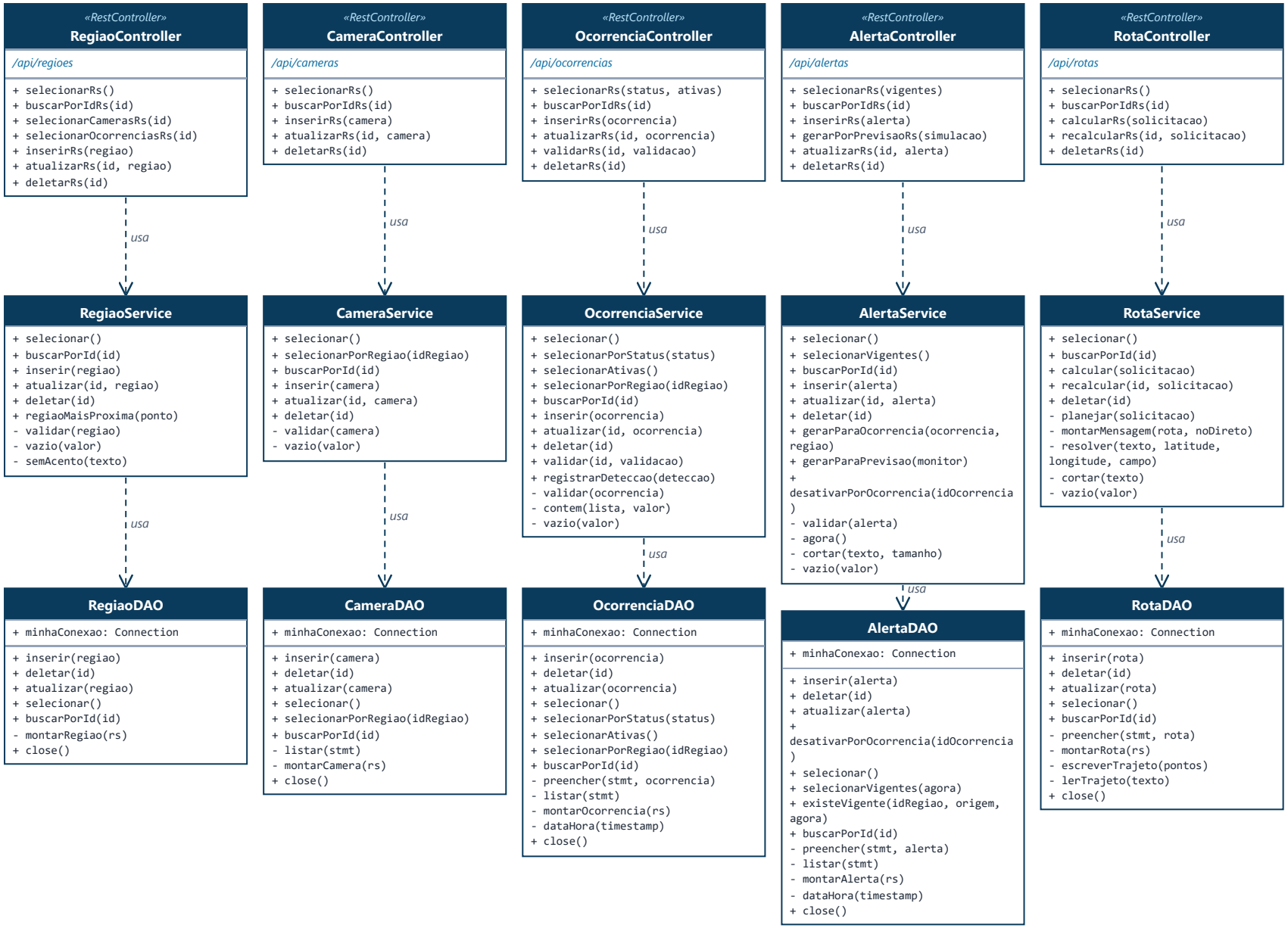
8. Diagrama de Classes

8.2 DTOs e exceções - objetos de entrada e saída da API e as exceções tratadas pelo `TratadorExcecoes`.



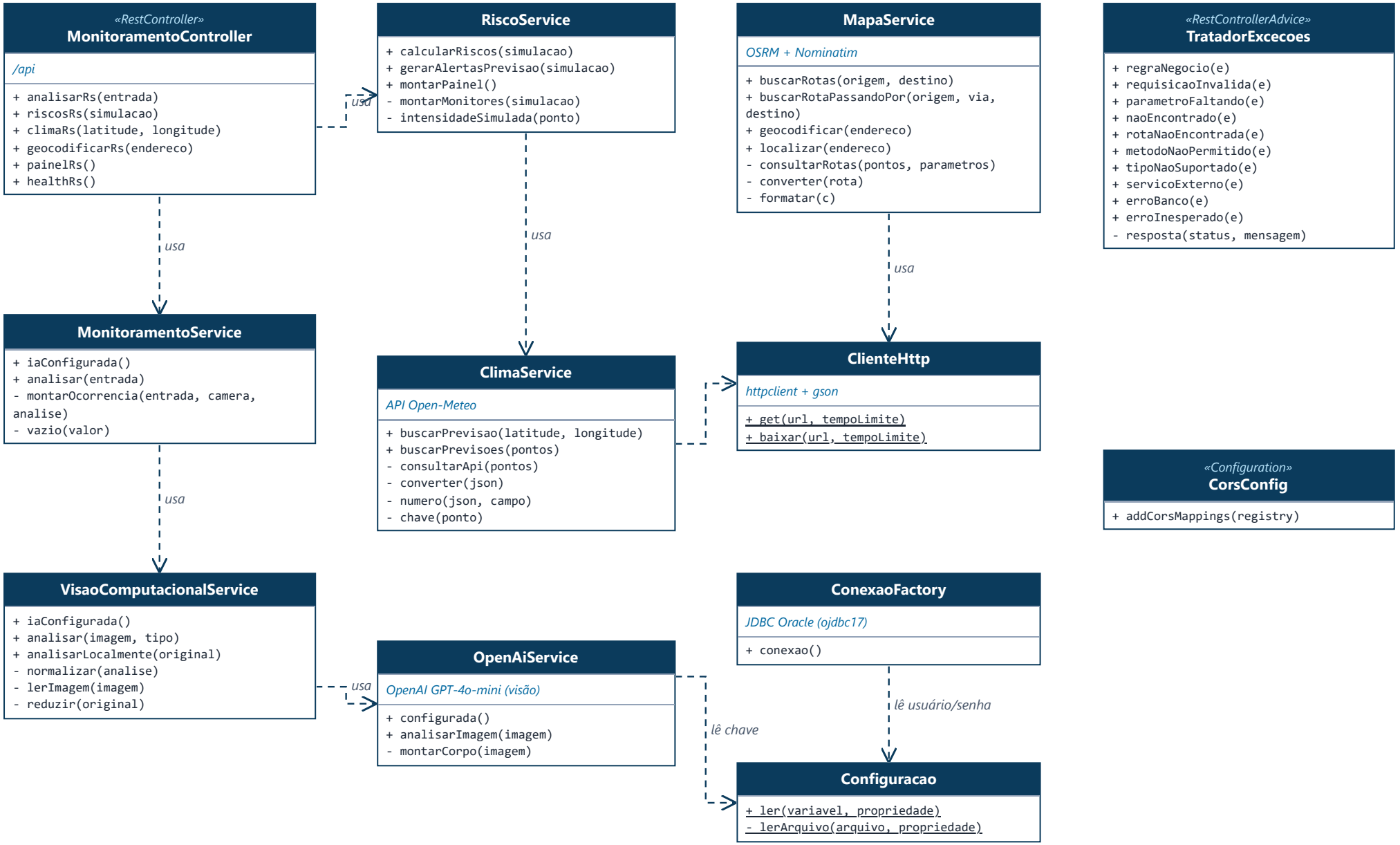
8. Diagrama de Classes

8.3 Camadas Controller, Service e DAO - cada recurso tem seu controller REST, um service com as validações (regras de negócio) e um DAO com JDBC puro (`PreparedStatement`) usando a `ConexaoFactory`.



8. Diagrama de Classes

8.4 Serviços de integração, IA e infraestrutura - previsão (Open-Meteo), mapas e rotas (OSRM / Nominatim), visão computacional (OpenAI ou motor local), conexão com o Oracle e tratamento de erros.



9. Como Executar o Projeto

Ferramentas e versões

Item	Versão / detalhe
Java JDK	21 (LTS)
Spring Boot	4.1.1 (spring-boot-starter-webmvc)
Maven	3.9 - já incluso no projeto pelo Maven Wrapper (`mvnw`), não precisa instalar
Banco de dados	Oracle Database da FIAP (`oracle.fiap.com.br:1521:orcl`), driver ojdbc17
Ferramentas	IntelliJ IDEA 2026.2 (back-end), VS Code (front-end), Oracle SQL Developer (script) e Postman/Insomnia (testes)
Front-end	Node.js 20 ou superior, Next.js 16.3.5, React 19, Tailwind CSS 3, Leaflet 1.9.4
Bibliotecas Java	Apache HttpClient 4.5.14 e Gson (chamadas às APIs externas)

Links

GitHub - API Java	https://github.com/Tuco2006/nexrain-springboot
GitHub - Front-end	https://github.com/Tuco2006/nexrain-front
Deploy - Site (Vercel)	https://nexrain-front.vercel.app/
Deploy - API (Render)	https://nexrain-springboot.onrender.com/api/health

A API usa o plano gratuito do Render, que desliga após um tempo sem uso: o primeiro acesso pode levar cerca de um minuto.

Passo a passo

1. **Banco:** conectar no Oracle da FIAP (DBeaver ou SQL Developer) e executar o arquivo `script.sql` inteiro como script (`Alt+X` no DBeaver, `F5` no SQL Developer). Ele cria as 5 tabelas e cadastra 12 regiões de São Paulo, 8 câmeras, 4 ocorrências e 2 alertas de exemplo.
2. **Credenciais:** usuário, senha e chave da OpenAI não ficam no código. Criar o arquivo `src/main/resources/local.properties` com `db.usuario=RM...`, `db.senha=...` e `openai.api.key=...` (a chave é opcional; sem ela a análise usa o motor local).
3. **API:** abrir a pasta `nexrain-springboot` no IntelliJ e rodar a classe `NexrainApplication`, ou no terminal: `.\mvnw spring-boot:run`. A API sobe em `http://localhost:8080`; conferir em `http://localhost:8080/api/health`.
4. **Front-end:** na pasta `nexrain-front`, criar o arquivo `.env.local` com `NEXT_PUBLIC_API_URL=http://localhost:8080` e rodar `npm install` e depois `npm run dev`. Acessar `http://localhost:3000`.
5. **Teste rápido:** em "Rota segura" clicar no exemplo "Sé → Shopping Ibirapuera" e ver o desvio do alagamento da Av. 23 de Maio; em "Monitoramento IA" analisar a CAM-01; em "Mapa de risco" ativar "Simular temporal".

Publicação

- **API no Render:** New > Web Service, conectar o repositório `nexrain-springboot`, linguagem **Docker** (usa o `Dockerfile` do projeto), Health Check Path `/api/health` e as variáveis `DB_USUARIO`, `DB_SENHA` e `OPENAI_API_KEY`.
- **Site na Vercel:** Add New > Project, importar o repositório `nexrain-front` e criar a variável `NEXT_PUBLIC_API_URL` com o endereço da API no Render (sem barra no final).

APIs externas usadas (todas gratuitas e sem chave, exceto a OpenAI): Open-Meteo (previsão), RainViewer (radar), OSRM (rotas), Nominatim / OpenStreetMap (endereços e mapa), Esri (mapa base).